

Serverless Contact Form with Amazon SES

Design, implementation and incident analysis for a static-site contact pipeline

Esmeralda Cabrera Ventura · esmeraldaspace.com
August 2026

About this document

This report documents the design and build of a serverless contact-form backend for a static portfolio site hosted on Amazon S3 and CloudFront. It is written for a public audience: every screenshot has been programmatically redacted to remove account identifiers, credentials, personal contact details and the unauthenticated endpoint address. The redaction method is described in Appendix A.

1. Executive summary

A static website cannot process a form. It has no server, no runtime and no way to hold a credential, so the conventional options are a third-party form service, a `mailto:` link, or a backend of one's own. This project takes the third route and builds the smallest backend that can correctly do one job: accept a contact form submission from one specific website and deliver it as email.

The result is four AWS components and roughly a hundred lines of client-side JavaScript. Amazon SES provides a verified sending identity. An AWS Lambda function written in Node.js receives the submission, validates it and calls SES. An IAM role grants that function exactly one permission. A Lambda Function URL exposes the function to the browser over HTTPS without an API Gateway in front of it. Configuration lives entirely in environment variables, so the deployed code contains no addresses and no secrets.

The build was not frictionless, and the report does not pretend otherwise. The first live submission failed with a browser CORS error caused by the same header being set in two places at once. Section 6 analyses that failure in detail, because the fix is less interesting than the reason the mistake was easy to make. A second, subtler issue is still open: the sending address belongs to a domain the account does not control, which undermines email authentication and pushes delivery toward spam folders. Section 9 sets out the remedy.

Scope of this document

Every figure in this report is a screenshot taken during the build. Figures appear in the order the work was carried out, with two exceptions noted in the captions where the original capture order did not match the logical sequence.

2. The problem

The portfolio site at `esmeraldaspace.com` is a static site: HTML, CSS and JavaScript files sitting in an S3 bucket and served through CloudFront. That architecture is fast, cheap and almost impossible to break, but it has one structural limitation. There is no process running anywhere that can receive an HTTP POST. A contact form on such a site therefore needs somewhere else to send its data.

Three approaches were considered.

- **A `mailto:` link.** Zero infrastructure, but it depends on the visitor having a configured desktop mail client. On a phone, or in a browser signed into webmail, it frequently does nothing at all. The visitor experiences this as a broken link, and the site owner never learns that anyone tried to make contact.
- **A third-party form service.** Fast to adopt and genuinely well engineered, but it introduces a dependency on an external company for a core function of the site, routes visitor messages through a third party, and typically requires a paid tier once volume or custom fields grow beyond a free allowance.
- **A self-hosted serverless backend.** More initial work, but the message path stays inside infrastructure the site owner controls, the running cost at portfolio volume is effectively zero, and the exercise produces transferable knowledge of SES, Lambda, IAM and CORS.

The third option was chosen. The deciding factor was not cost, since all three are close to free at this volume, but control and learning value. A contact form is a low-stakes place to make mistakes with IAM policies and cross-origin headers, and the mistakes made here are documented in Section 6 precisely because they are instructive.

The requirements were modest and were fixed before any resource was created. The form must deliver a message to a single inbox. It must accept submissions only from the portfolio's own origin. It must never silently lose a message. It must hold no credentials in client-side code, which is public by definition. And it must be cheap enough to leave running indefinitely without thinking about it.

3. Architecture

The complete request path involves four AWS services and one round trip from the browser.

1	Visitor completes the form at <code>esmeraldaspace.com/contact.html</code>
2	<code>contact.js</code> serialises the fields and POSTs JSON to the Lambda Function URL
3	Lambda Function URL performs the CORS preflight check and invokes the function
4	The handler validates the payload and calls the SES SendEmail API
5	SES delivers the message to the verified recipient address
6	The handler returns a JSON body containing an explicit success flag
7	<code>contact.js</code> reads that flag and shows either confirmation or a recovery panel

Two properties of this path are worth drawing out, because both were deliberate and both are discussed further in Section 4.

First, there is no API Gateway. A Lambda Function URL is a built-in HTTPS endpoint attached directly to the function. For a single-route, single-method endpoint it removes an entire service from the design without giving up anything this project needs.

Second, the last two steps are not redundant. An HTTP 200 status means the function ran and returned, not that an email was accepted for delivery. The handler therefore reports its own outcome inside the response body, and the client checks that field rather than the status code. This distinction is the difference between a form that is honest and a form that lies politely.

4. Design decisions and their trade-offs

Each decision below is recorded with the reasoning that produced it and the cost that came with it. Where a decision is known to be imperfect, that is stated rather than glossed.

4.1 Amazon SES rather than an SMTP relay

SES is an API-first sending service that lives in the same account and IAM model as the rest of the infrastructure. That matters more than it first appears: the Lambda function can be granted permission to send email through its execution role, using temporary credentials issued by AWS, with no password stored anywhere. An SMTP relay would require a username and password to be held somewhere and rotated by hand. Removing a long-lived credential from the design removes an entire category of failure.

The trade-off is the SES sandbox. New accounts may send only to verified addresses until production access is granted. For this design the destination is a single verified inbox, so the sandbox is not a constraint today; it becomes one the moment the system needs to email the visitor, for example to acknowledge receipt.

4.2 Email-address identities rather than a domain identity

SES will verify either a whole domain or a single email address. Address verification is faster: SES sends a confirmation link, the owner clicks it, and the identity is live in under a minute. Domain verification requires publishing DNS records and waiting for propagation.

Address verification was chosen to reach a working end-to-end path quickly. This is the weakest decision in the build, and the console warned about it at the time. The consequences are analysed in Section 9.2; the short version is that it makes the outgoing mail fail domain authentication alignment and therefore look suspicious to receiving mail servers. It was the right call for a first iteration and is the first thing that should change.

4.3 A Lambda Function URL rather than API Gateway

API Gateway offers request validation, throttling, usage plans, custom domains, WAF integration and staged deployments. This endpoint needs none of them. It has one route, one method and one consumer. A Function URL provides an HTTPS endpoint with built-in CORS configuration and nothing else, which is an accurate match for the requirement.

The costs of that simplicity are real and worth naming. There is no built-in rate limiting, so abuse protection must come from Lambda reserved concurrency or from logic inside the handler. There is no WAF in front of the endpoint. The URL is an AWS-generated hostname rather than a custom domain. And because the auth type is NONE, the endpoint is reachable by anyone who learns the address, so the CORS configuration is a browser-side convenience, not a security boundary. Section 7 addresses that distinction directly.

4.4 Configuration in environment variables

Three values that would otherwise be hard-coded are supplied to the function as environment variables: the origin permitted to call it, the address messages are sent from, and the address messages are delivered to. None is a secret in the cryptographic sense, but keeping them out of the source has three practical benefits. The recipient can be changed without redeploying code. The same function can be pointed at a staging origin. And the source can be published or shared without exposing personal addresses.

Lambda encrypts environment variables at rest with a service-managed key by default. For genuine secrets, such as an API token, this would not be sufficient on its own and Secrets Manager or SSM Parameter Store would be the correct home. For configuration of this kind it is appropriate.

4.5 An inline IAM policy granting exactly one action

Lambda creates an execution role automatically, and that role begins with permission to write CloudWatch Logs and nothing more. Sending email requires an additional grant. The temptation at this point is to attach a broad managed policy such as AmazonSESFullAccess, which would work immediately and would also allow the function to delete identities, alter the suppression list and read sending statistics.

Instead a hand-written inline policy grants a single action: `ses:SendEmail`. If the function were ever compromised through a payload-handling bug, the attacker inherits the ability to send email and nothing else. This is the least-privilege principle applied at the smallest useful granularity, and it costs about ninety seconds of extra work.

The policy as written specifies a resource wildcard. Tightening it is discussed in Section 9.3.

4.6 Treating HTTP 200 as insufficient evidence of delivery

This is the design decision with the most thought behind it and the least visibility in the infrastructure. A Lambda function that catches an SES exception and returns normally will still produce a 200 response. A client that treats 200 as success will then tell the visitor their message was sent when it was not. The visitor closes the tab and the message is gone.

The handler therefore reports its own outcome in a success field inside the response body, and the client checks that field. When delivery fails, the form does not show a generic error. It states plainly that the message did not go through, confirms that nothing the visitor typed has been lost, and offers a pre-filled mail draft containing their message along with a fallback address they can write to directly.

This is graceful degradation applied to a contact form. The failure mode is designed with as much care as the success path, on the reasoning that a contact form which quietly loses messages is worse than no contact form at all, because it also consumes the visitor's goodwill. Figure 31 shows this path working during the incident described in Section 6, which is the only reason that incident cost nothing.

5. Implementation walkthrough

The build proceeded in seven phases. Each phase below opens with the reasoning behind it, followed by the screenshots taken while it was carried out. Captions record what the figure shows and, where relevant, which design decision it illustrates.

5.1 Phase 1. Establishing a verified sending identity

SES will not send mail from an address it has not confirmed the owner controls. This phase establishes that proof for two addresses: the one that will send and receive production messages, and a second used to submit test messages from a different mailbox.

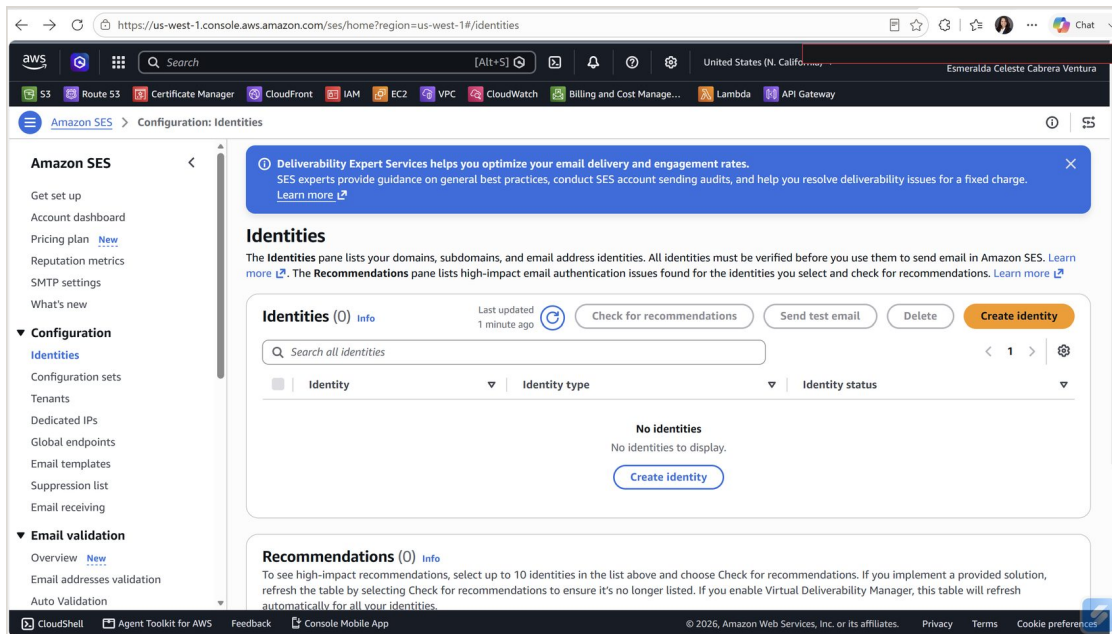


Figure 1. The SES Identities console in the US West (N. California) region before any work begins. No identities exist, so the account cannot yet send mail from any address.

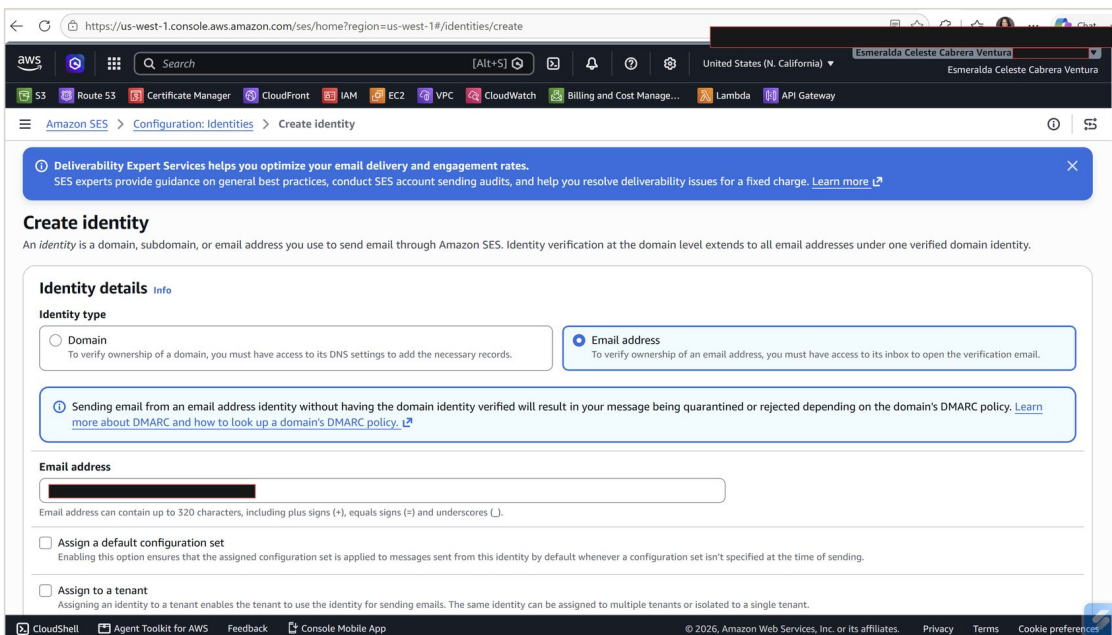


Figure 2. Creating the first identity. Address verification is selected over domain verification for speed. The console warning visible here is significant: sending from an address identity without a verified domain can cause messages to be quarantined or rejected under the receiving domain's DMARC policy. Section 9.2 returns to this.

Create identity

An identity is a domain, subdomain, or email address you use to send email through Amazon SES. Identity verification at the domain level extends to all email addresses under one verified domain identity.

Identity details info

Identity type

☐ Domain
To verify ownership of a domain, you must have access to its DNS settings to add the necessary records.

☒ Email address
To verify ownership of an email address, you must have access to its inbox to open the verification email.

Warning: Sending email from an email address identity without having the domain identity verified will result in your message being quarantined or rejected depending on the domain's DMARC policy. [Learn more about DMARC and how to look up a domain's DMARC policy.](#)

Email address

Email address can contain up to 320 characters, including plus signs (+), equals signs (=), and underscores (_).

☐ Assign a default configuration set
Enabling this option ensures that the assigned configuration set is applied to messages sent from this identity by default whenever a configuration set isn't specified at the time of sending.

☐ Assign to a tenant
Assigning an identity to a tenant enables the tenant to use the identity for sending emails. The same identity can be assigned to multiple tenants or isolated to a single tenant.

Tags - optional info

You can add one or more tags to help manage and organize your resources, including identities.
No tags associated with the resource.

[Add new tag](#)
You can add up to 50 tags.

[Cancel](#) [Create identity](#)

Figure 3. The completed identity form before submission. No configuration set or tenant is assigned, since neither is needed for a single low-volume sender.

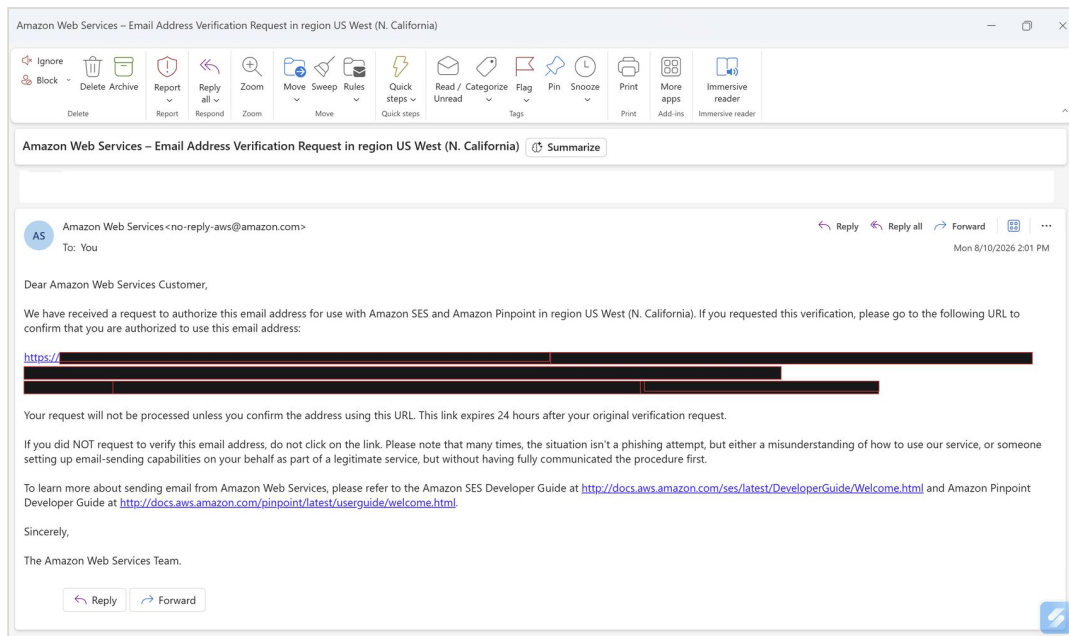


Figure 4. The verification request arriving in the destination mailbox. The confirmation link has been redacted in full: it carries an access key identifier and a request signature as query parameters.

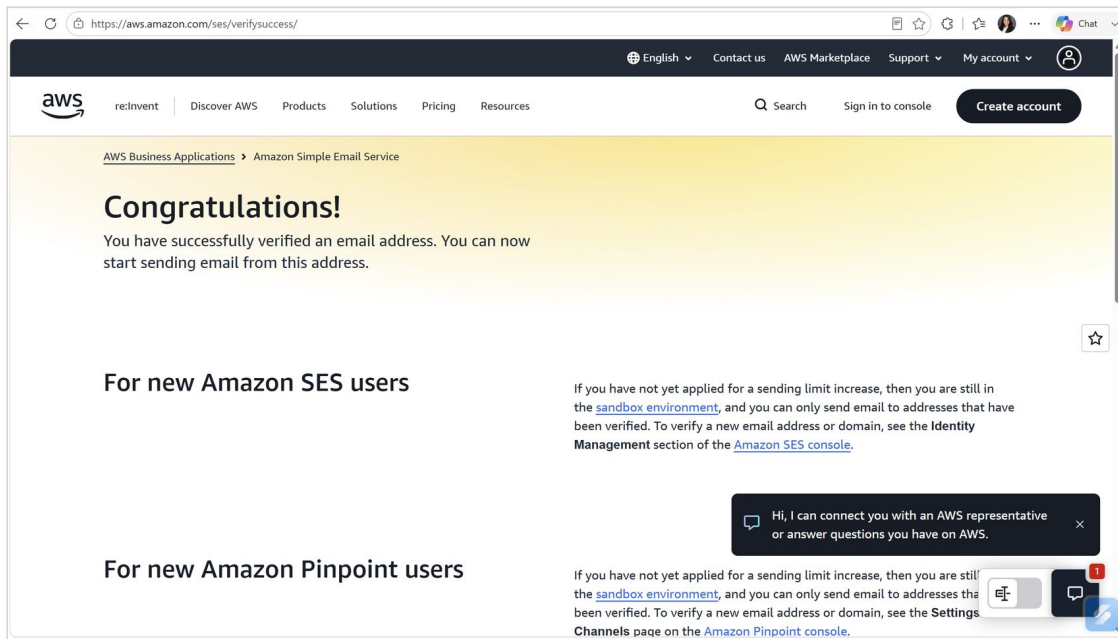


Figure 5. Confirmation that the first address is verified. The page notes that new accounts remain in the SES sandbox and may send only to verified addresses, which is the constraint discussed in Section 4.1.

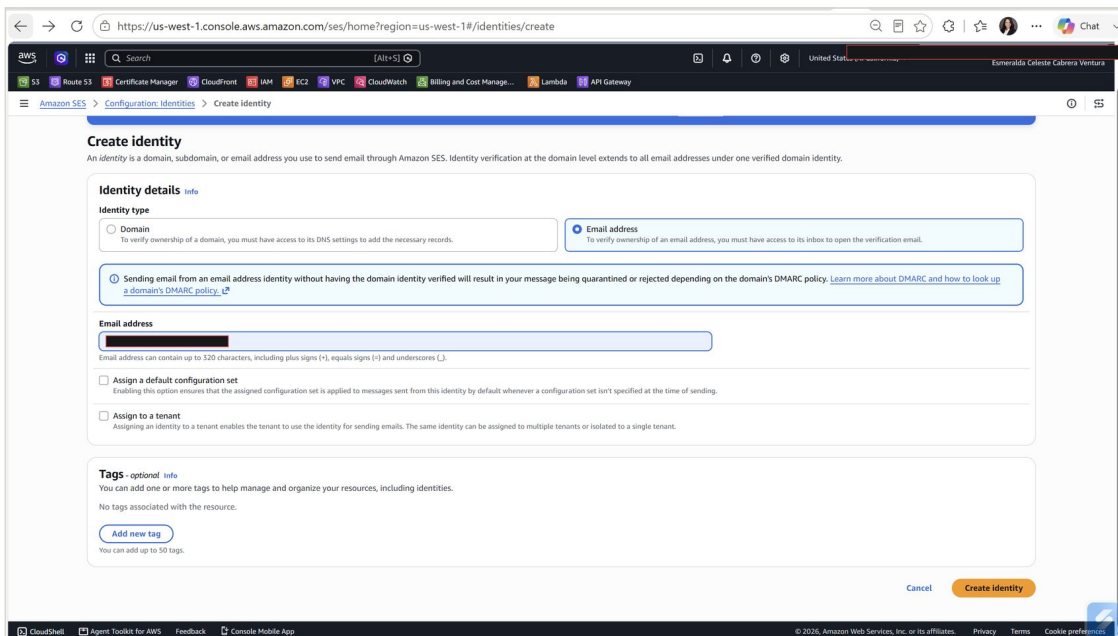


Figure 6. A second identity is created for a different mailbox, used to submit test messages from an address other than the recipient.

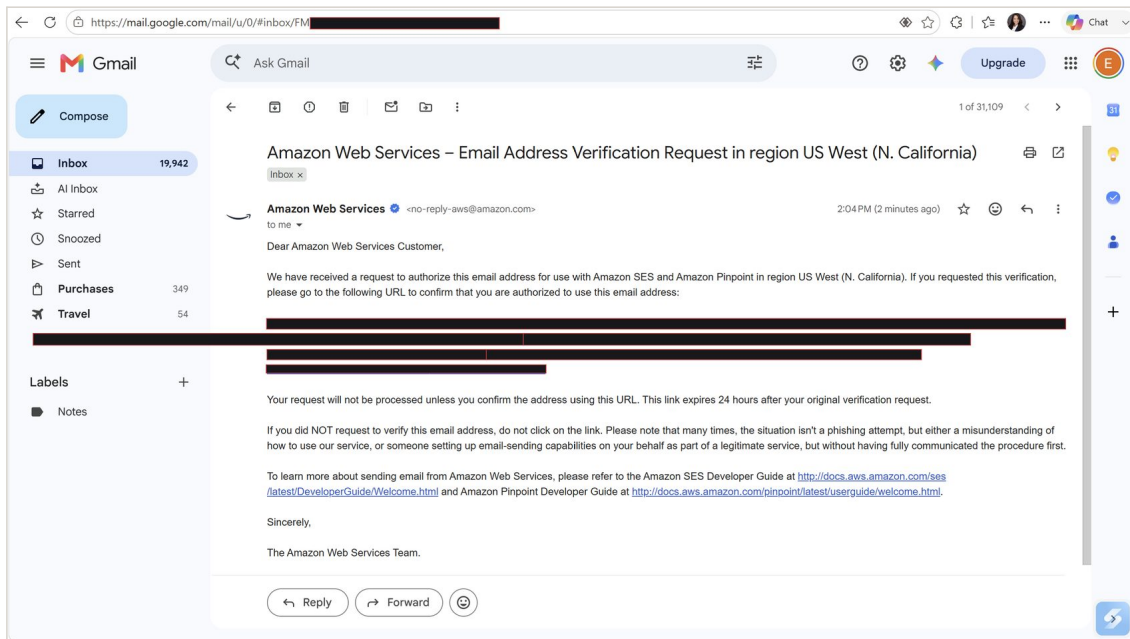


Figure 7. The second verification request, this time in a webmail client. The link is redacted for the same reason as Figure 4.

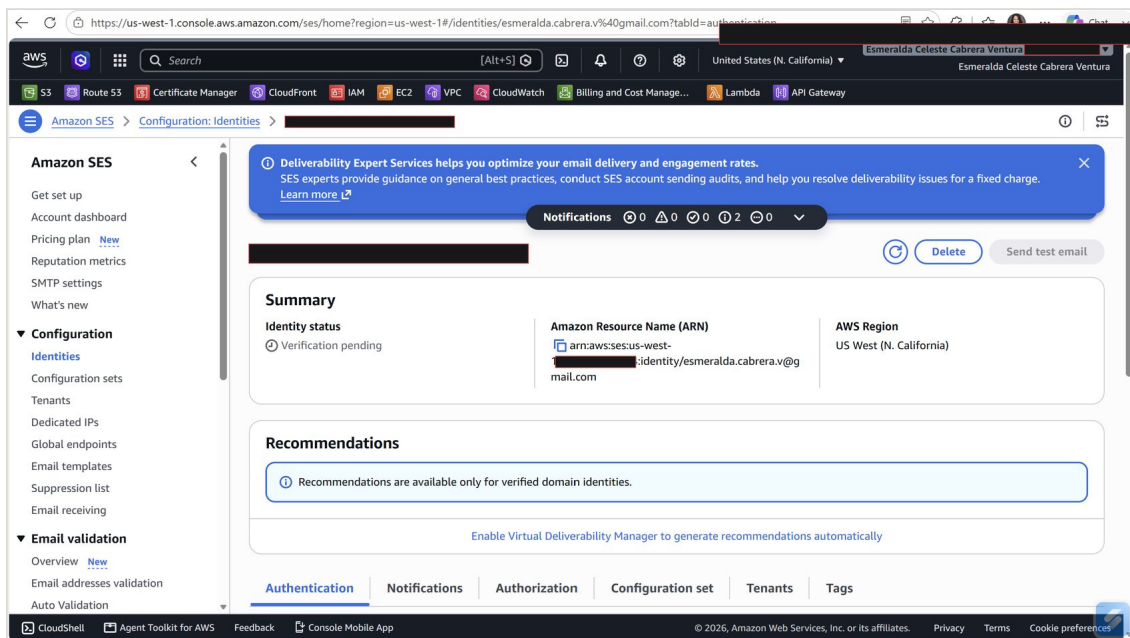


Figure 8. The identity detail view while verification is still pending, showing the identity ARN and the region. Recommendations are unavailable because they apply only to verified domain identities, a further consequence of the choice made in Figure 2.

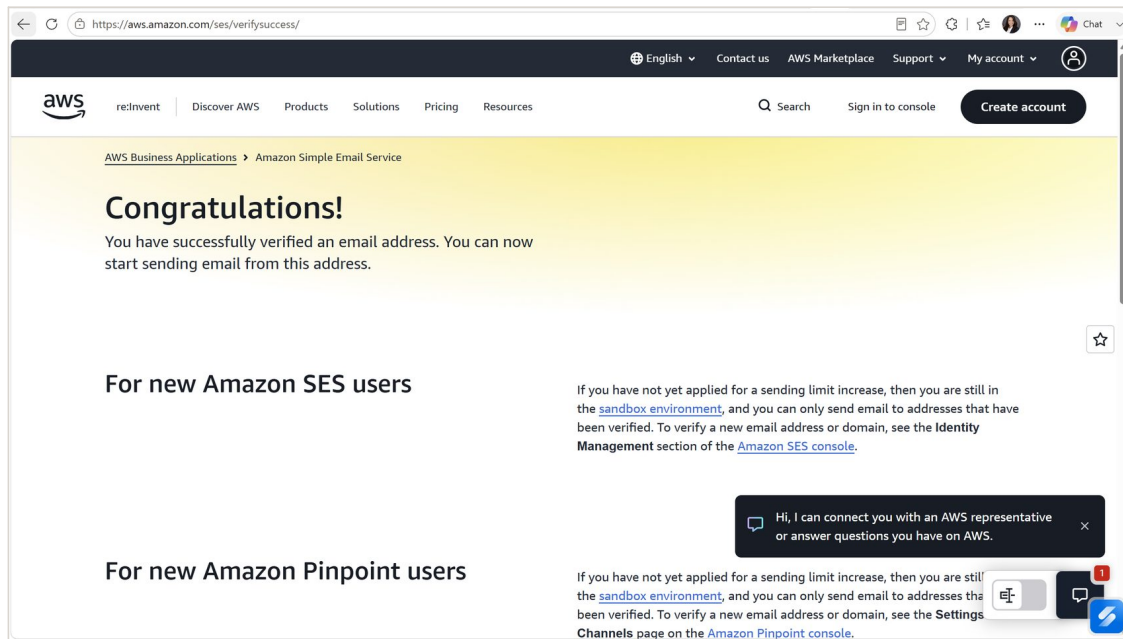


Figure 9. The second address is verified. Both the sending and testing identities are now live.

5.2 Phase 2. Creating the function

With a verified identity in place, the compute layer is created: a single Node.js function authored from scratch rather than from a blueprint, since the logic is short and specific.

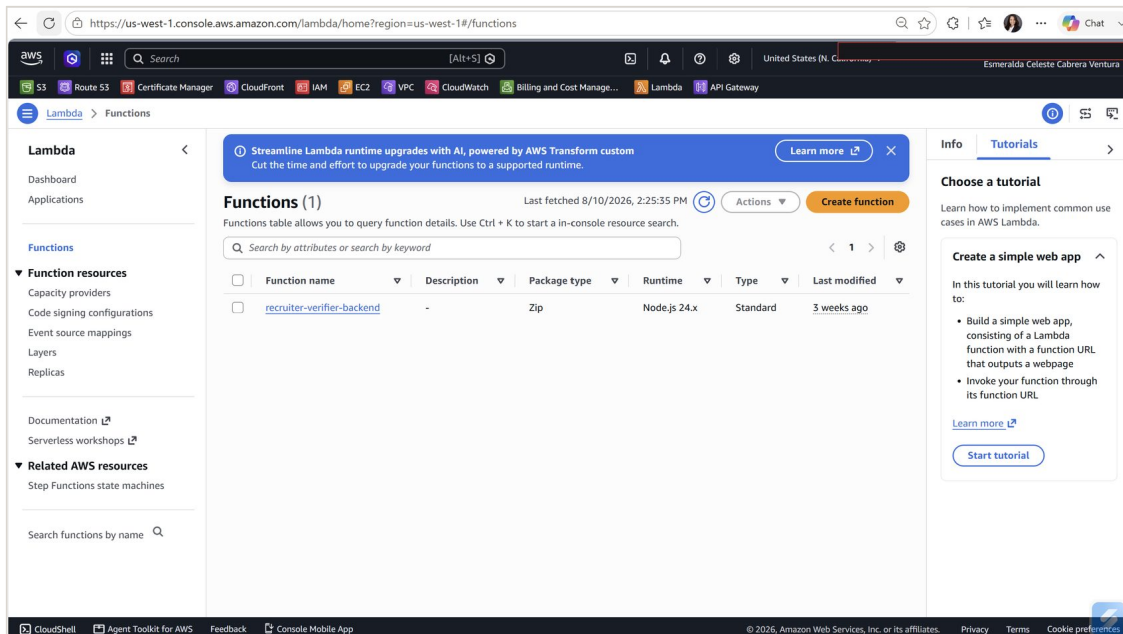


Figure 10. The Lambda console before the contact form function is created. One unrelated function already exists in the account.

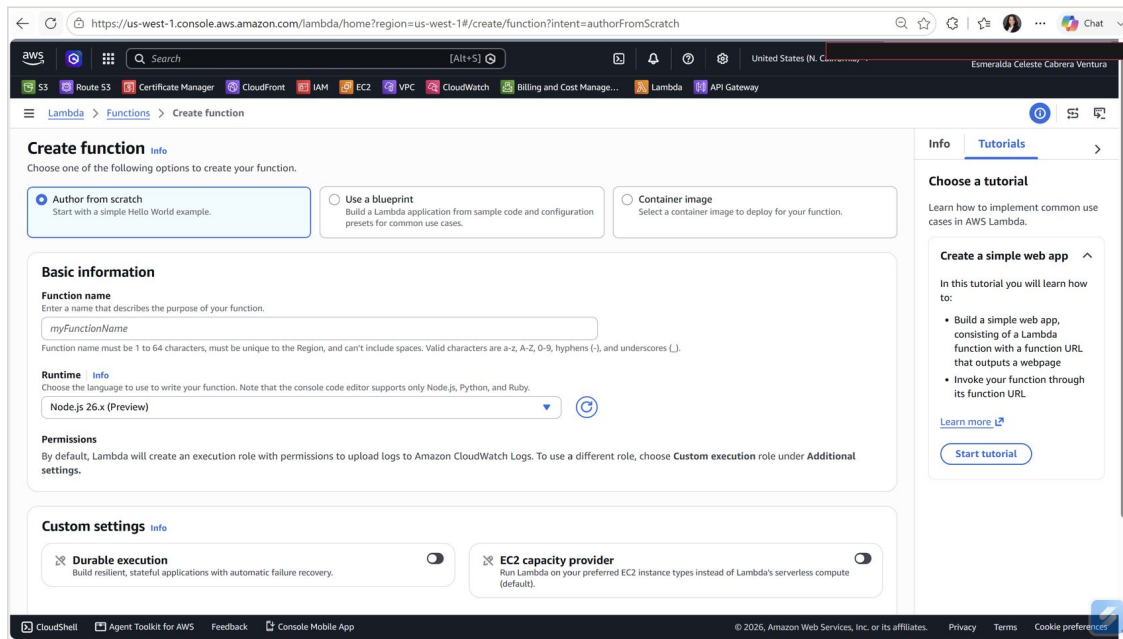


Figure 11. Authoring from scratch rather than from a blueprint or a container image. The logic is short and specific enough that a template would add more to remove than to keep.

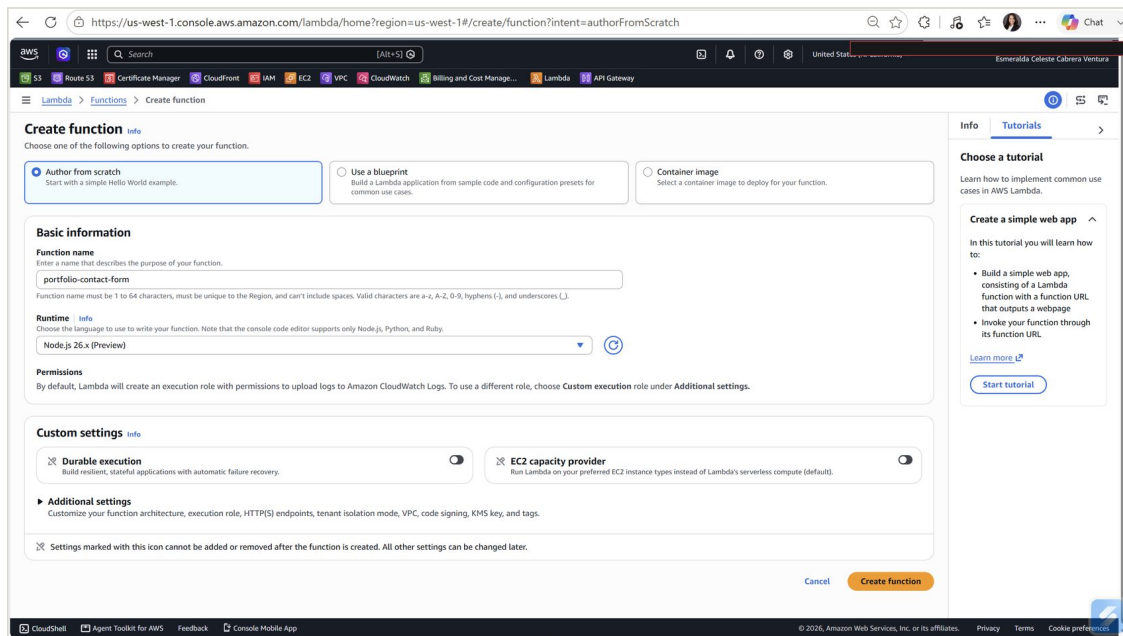


Figure 12. The function is named portfolio-contact-form. Note the runtime offered by the console at this point is a preview release; Section 9.4 recommends pinning a generally available version.

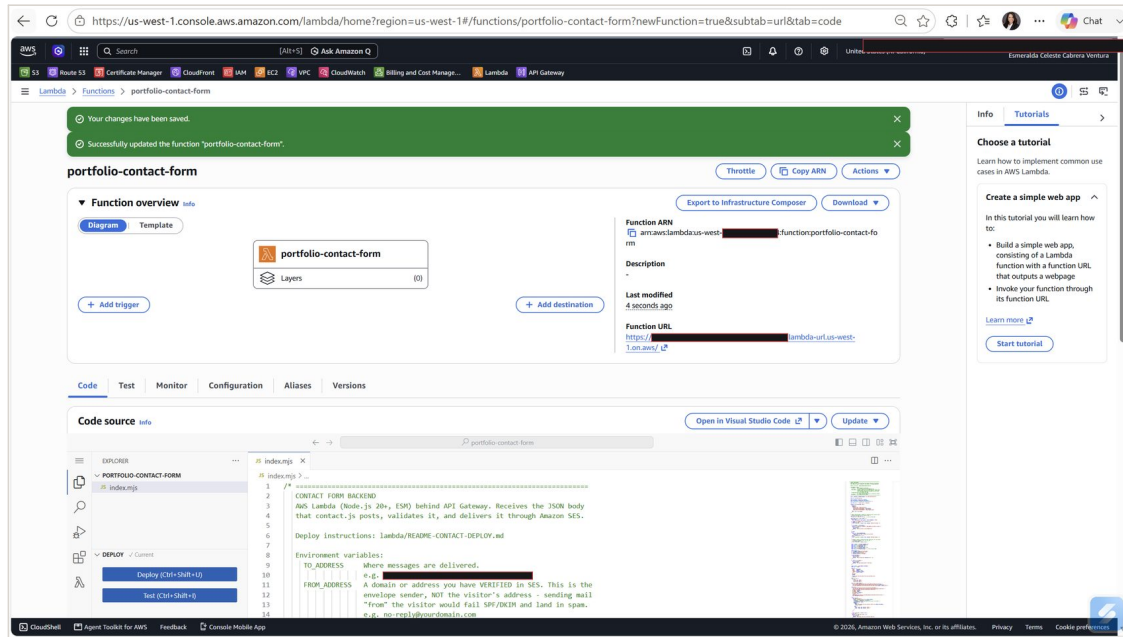


Figure 13. The deployed function. The header comment in `index.mjs` documents the environment variables the handler expects and states explicitly that the From address must be an identity verified in SES, because sending as the visitor's own address would fail SPF and DKIM and land in spam. The comment's reference to API Gateway is a leftover from an earlier plan and no longer describes the deployment.

5.3 Phase 3. Lifting configuration out of the code

Three values that would otherwise be embedded in the source are supplied as environment variables, so that the deployed code contains no addresses and the recipient can be changed without a redeployment.

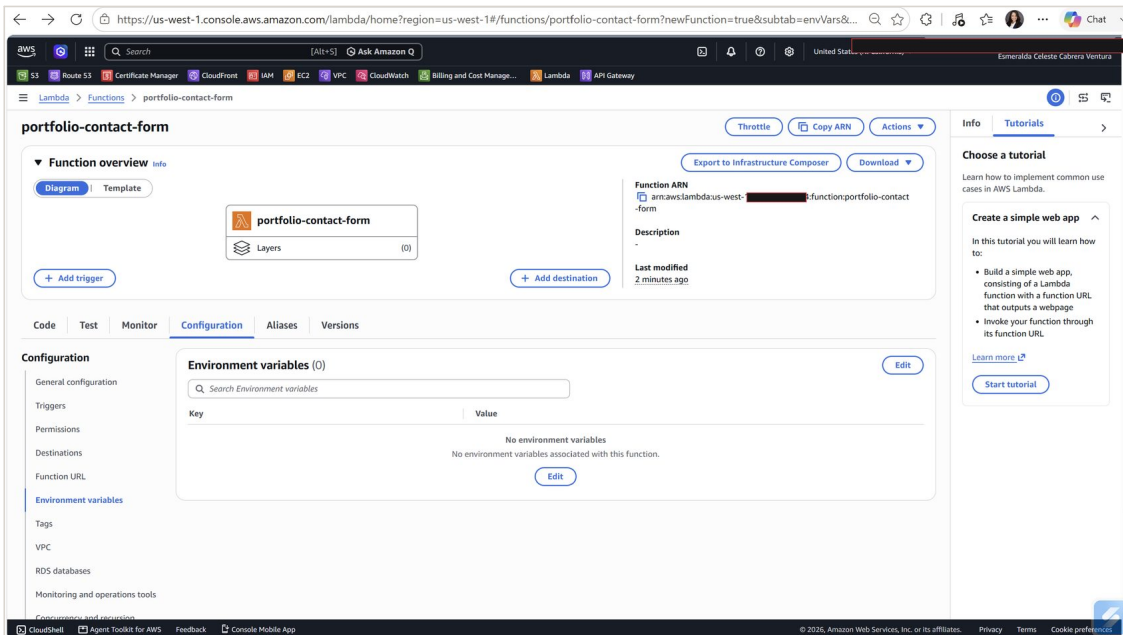


Figure 14. The function's environment variables before configuration: none are set.

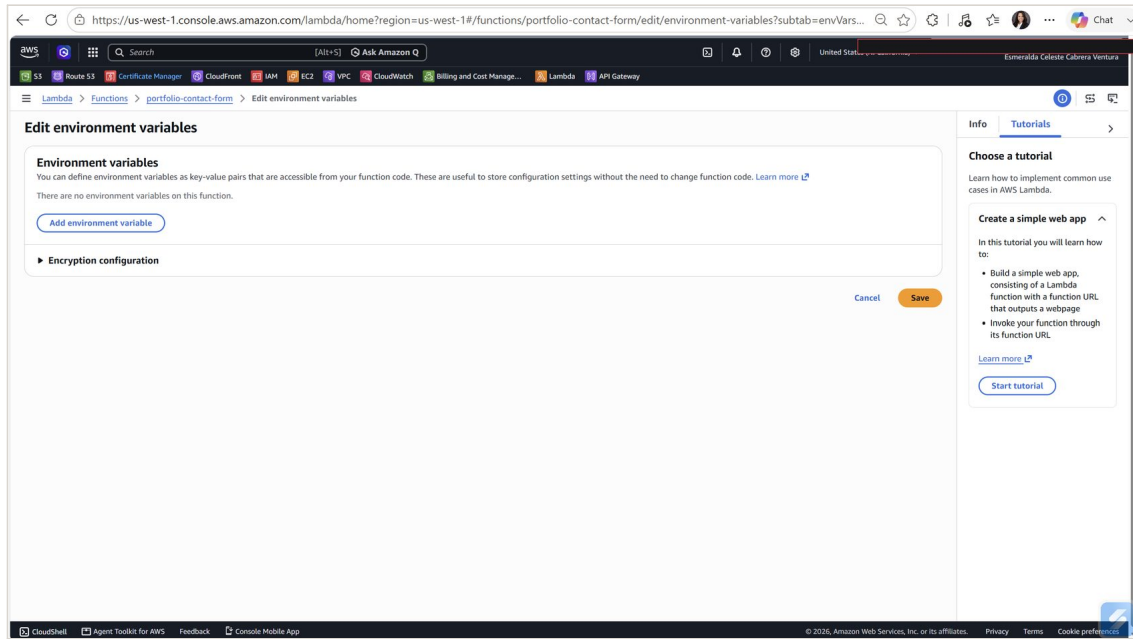


Figure 15. The environment variable editor, empty.

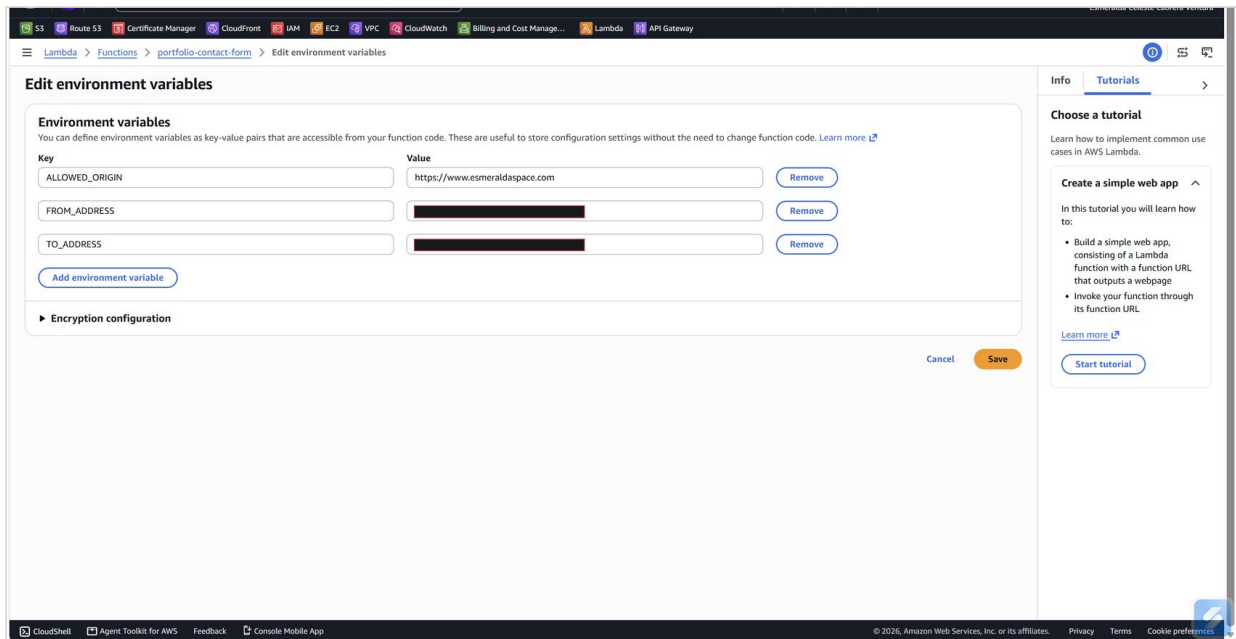


Figure 16. The three configuration values entered: the origin permitted to call the function, the address mail is sent from, and the address mail is delivered to. Both addresses are redacted here.

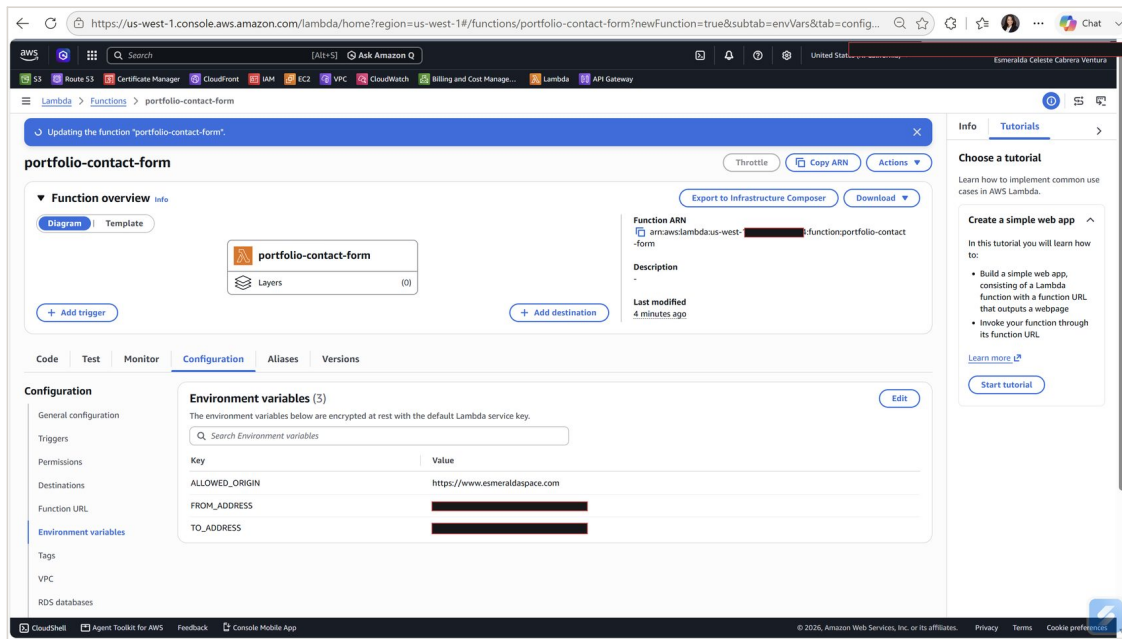


Figure 17. Configuration saved. The console confirms that environment variables are encrypted at rest with the default Lambda service key.

5.4 Phase 4. Granting exactly one permission

The execution role starts with logging rights only. Rather than attaching a broad managed policy, a hand-written inline policy adds the single action the function needs.

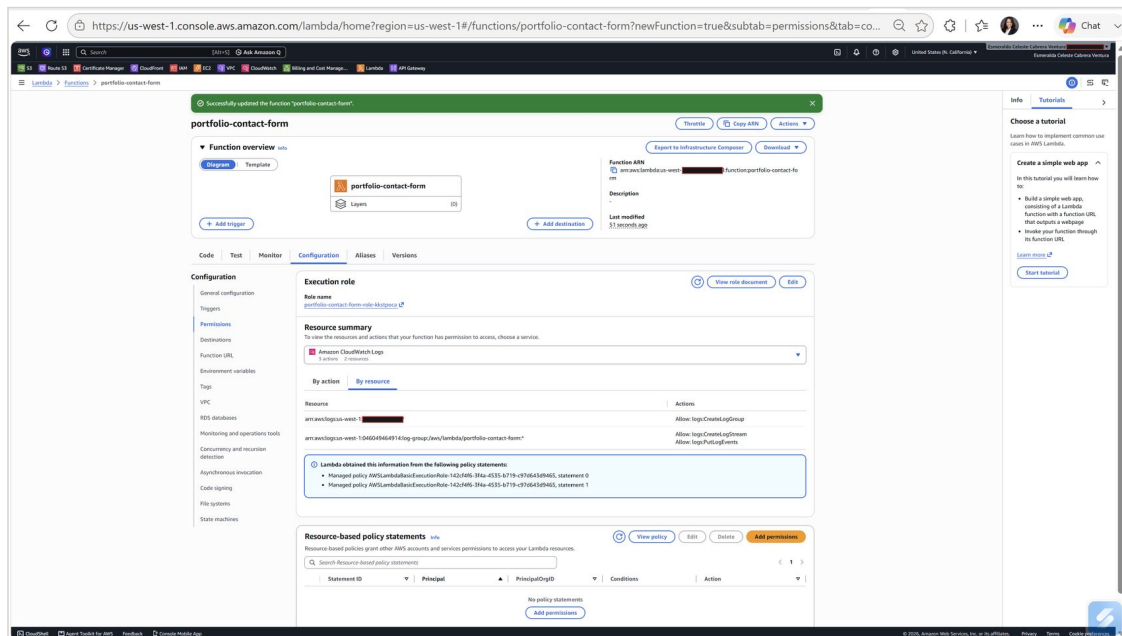


Figure 18. The execution role as Lambda created it. Its only permissions are the three CloudWatch Logs actions needed to write its own logs. It cannot yet send email.

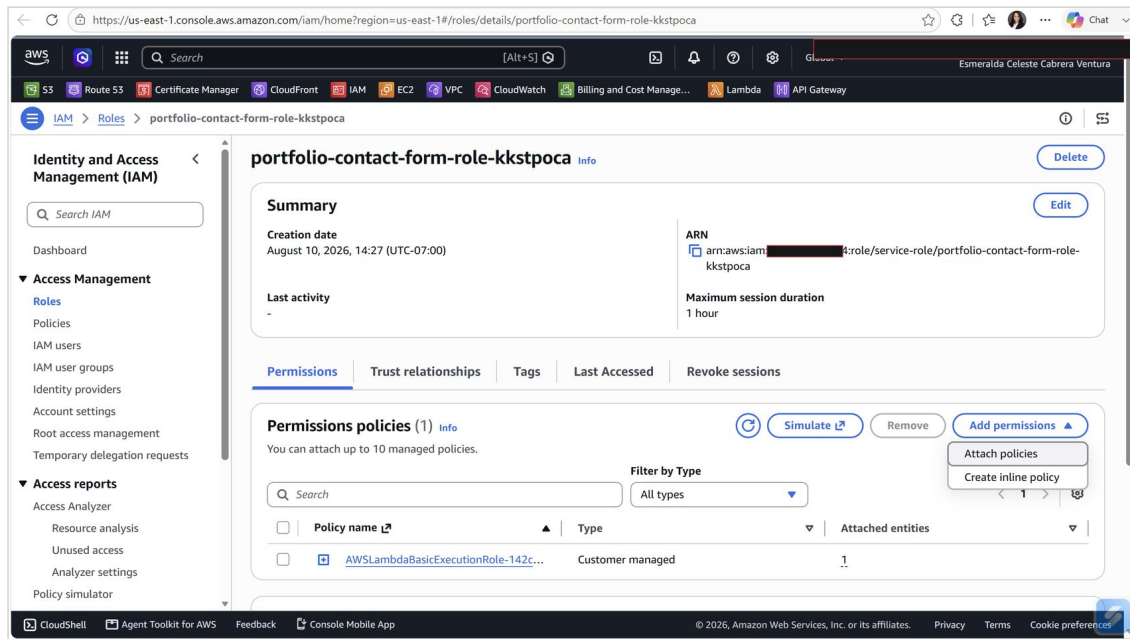


Figure 19. The role in the IAM console, about to receive an additional permission. The choice at this point is between attaching a broad managed policy and writing a narrow inline one.

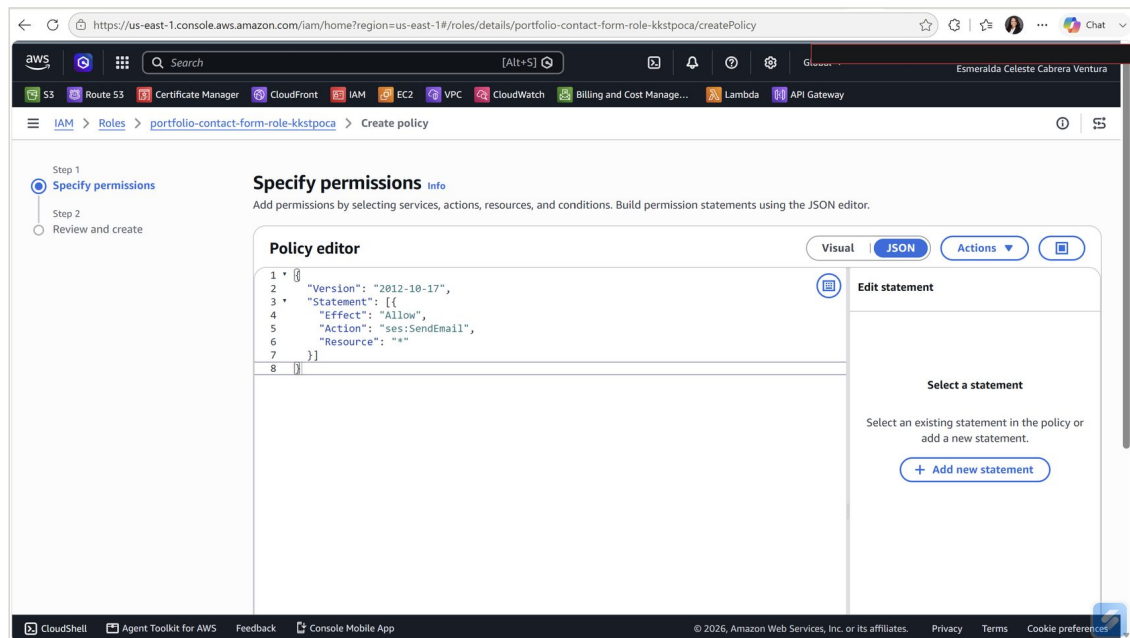


Figure 20. The inline policy, written directly as JSON. It grants a single action: ses:SendEmail. This is the least-privilege decision described in Section 4.5.

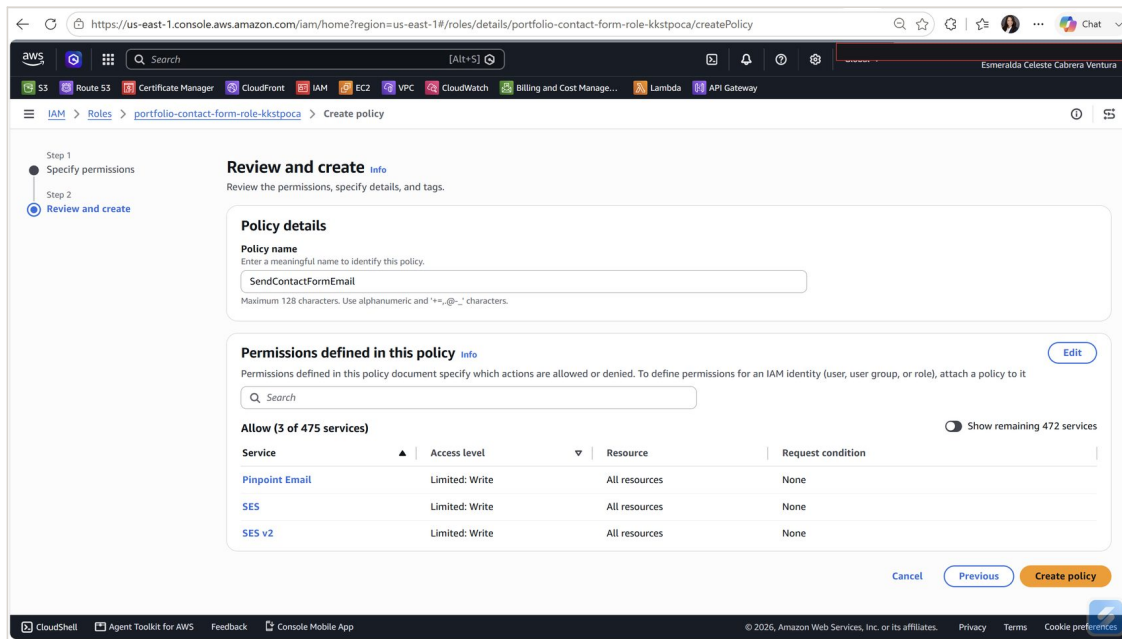


Figure 21. The review step, with the policy named SendContactFormEmail. IAM reports the grant as affecting three related services, because the single SendEmail action is surfaced across the SES, SES v2 and Pinpoint Email APIs. The access level remains limited to write.

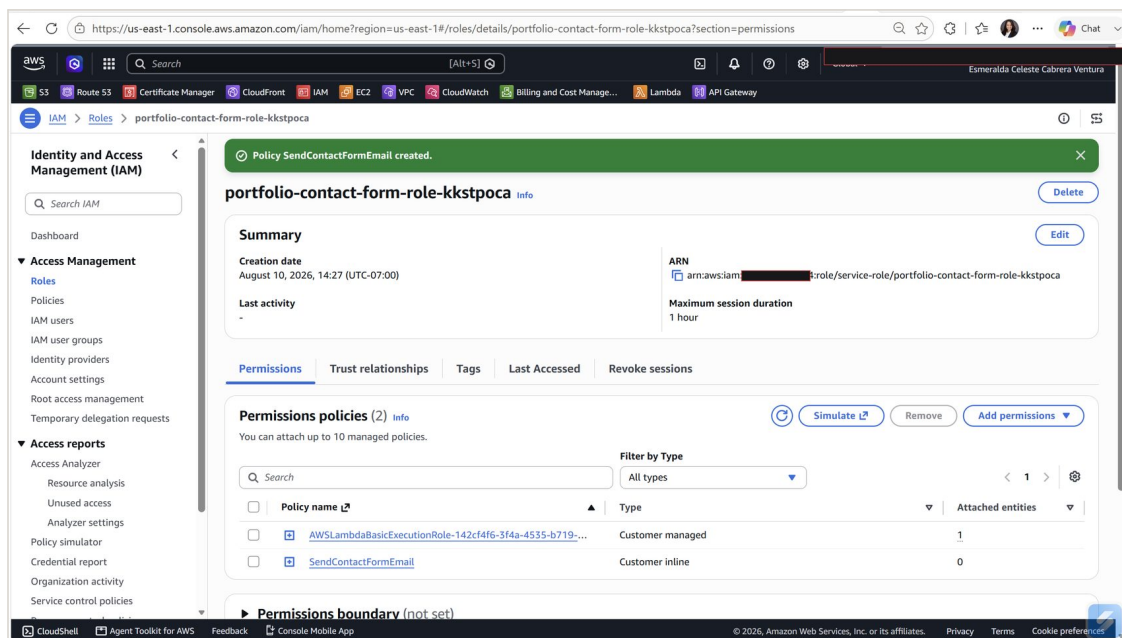


Figure 22. The policy is attached. The role now holds two policies: the original logging permissions and the new single-action send permission.

5.5 Phase 5. Exposing the function to the browser

A Function URL gives the handler an HTTPS endpoint. This phase also introduces the CORS configuration that later proves to be one layer too many.

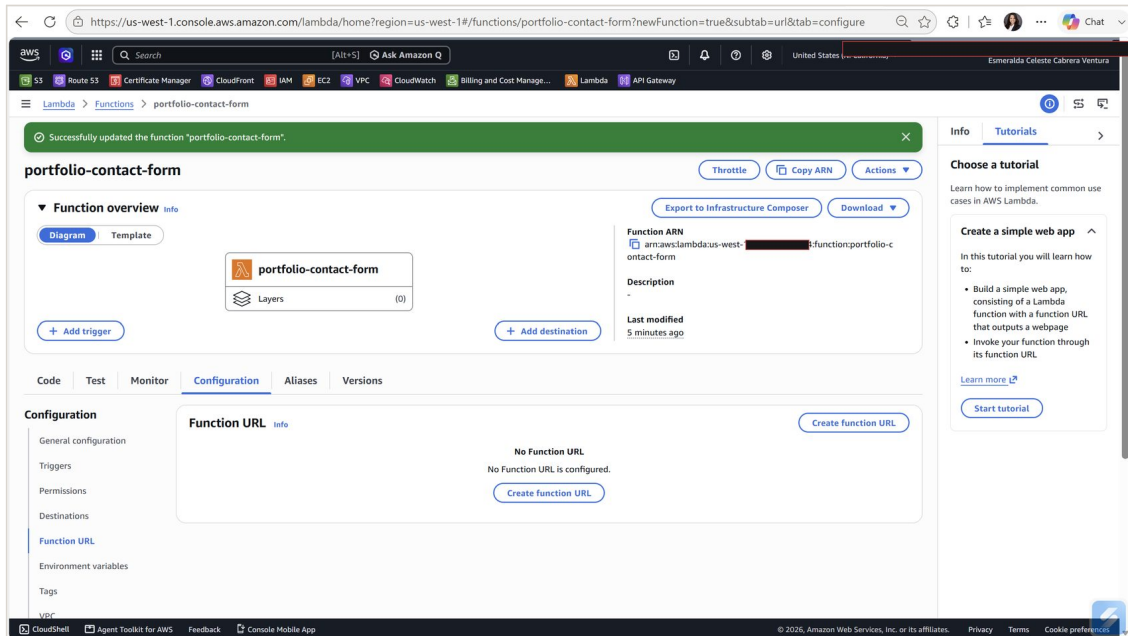


Figure 23. The function has no HTTPS endpoint yet, so the browser has no way to reach it.

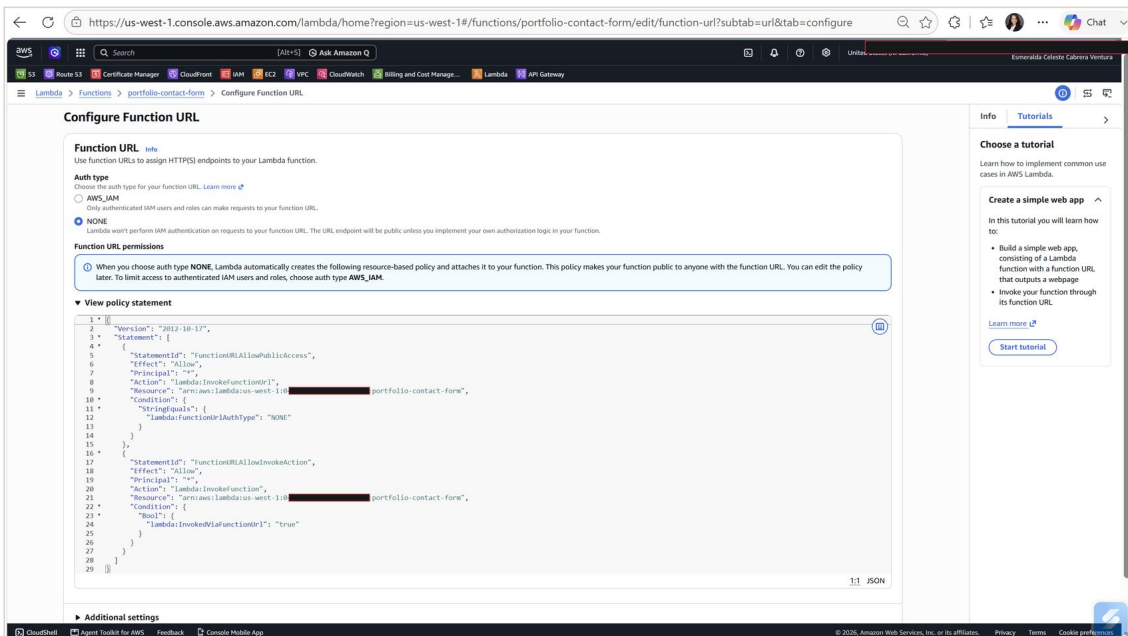


Figure 24. Configuring the Function URL with an auth type of NONE. The console generates the resource-based policy shown and states plainly that the function becomes reachable by anyone with the URL. Section 7 discusses why this risk was accepted and how the blast radius was limited.

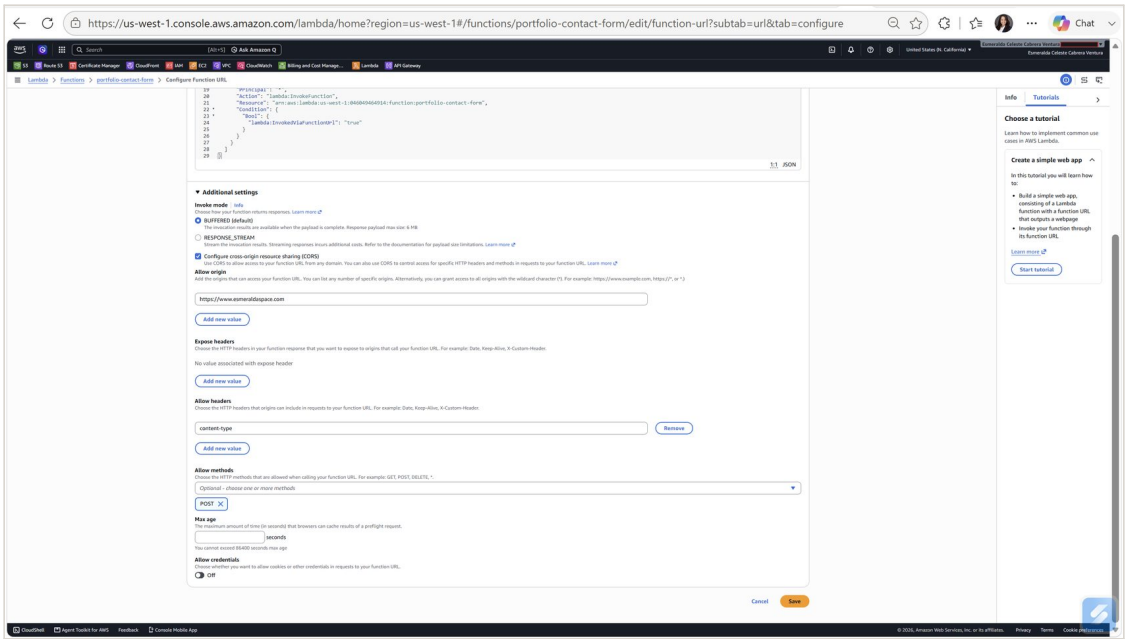


Figure 25. The CORS settings on the Function URL: a single permitted origin, the content-type header allowed, the POST method only, and credentials disabled. Every value here is correct in isolation. The problem, diagnosed in Section 6, is that the handler is setting the same origin header itself.

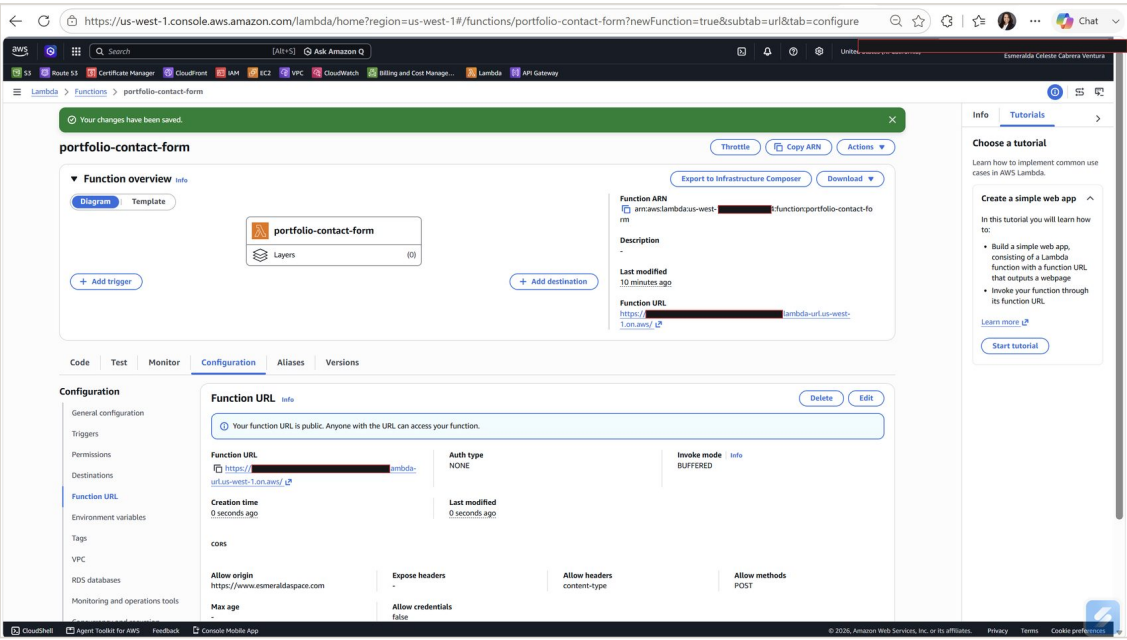


Figure 26. The endpoint is live, with its CORS configuration summarised. The hostname is redacted throughout this report because the endpoint is unauthenticated.

5.6 Phase 6. Deploying the front end

The client script is pointed at the endpoint, uploaded to the site bucket, and the CDN cache is invalidated so that visitors receive the new file rather than the cached previous version.

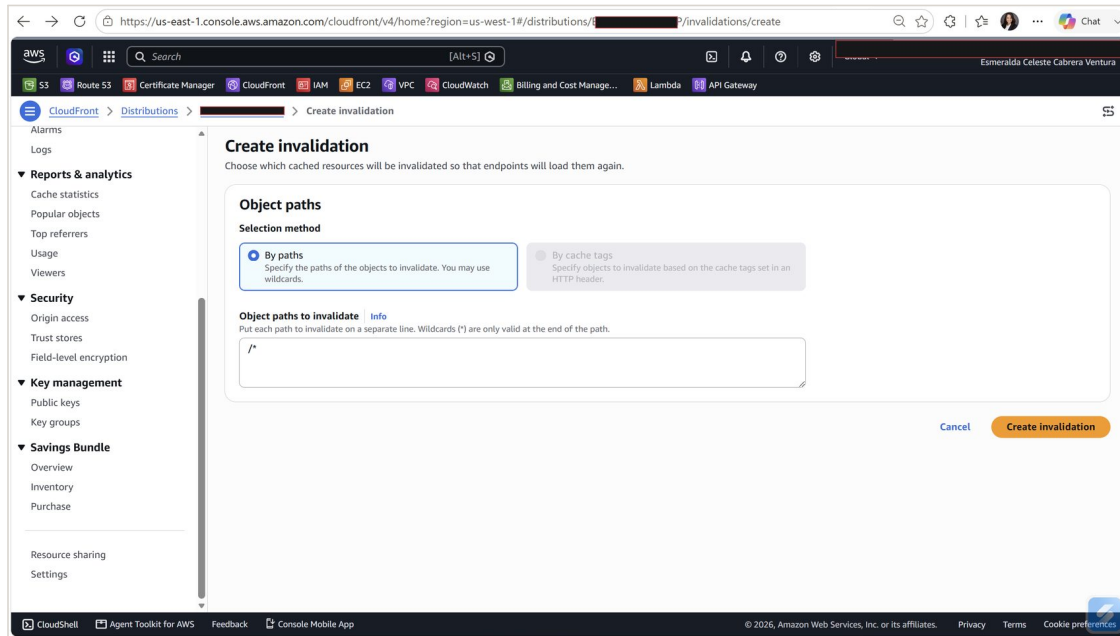


Figure 27. Creating a CloudFront invalidation across all paths. Without this, visitors continue to receive the cached previous version of the client script. Captured slightly ahead of its place in the sequence; the deployment loop is edit, upload, then invalidate.

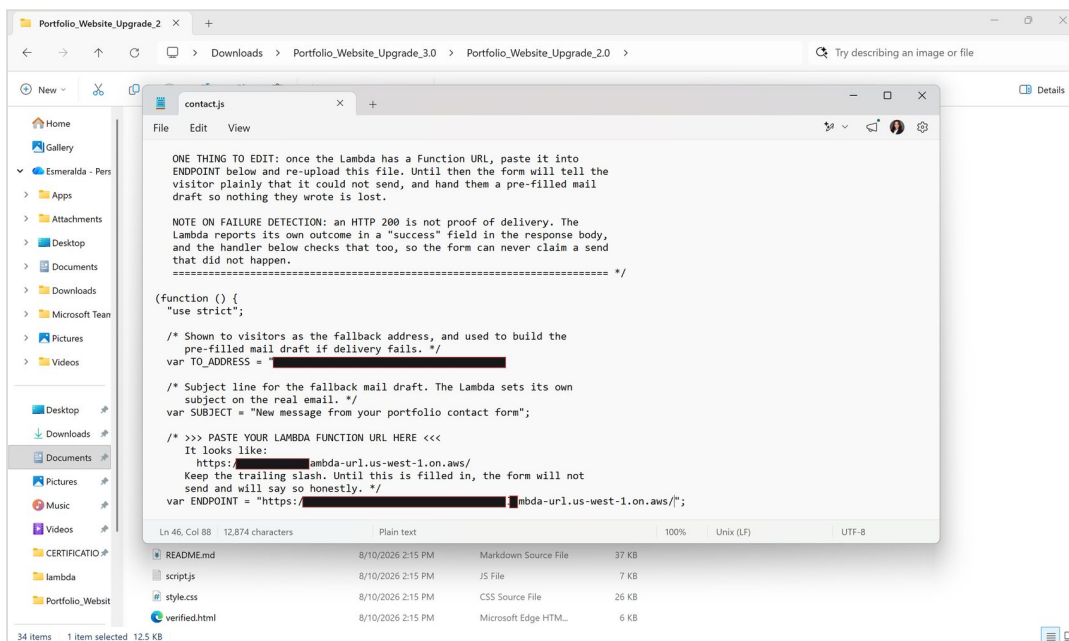


Figure 28. The client script with the endpoint filled in. The surrounding comments record two deliberate decisions: until the endpoint is configured the form tells the visitor plainly that it cannot send rather than failing silently, and an HTTP 200 is explicitly not treated as proof of delivery. The endpoint and the fallback address are redacted.

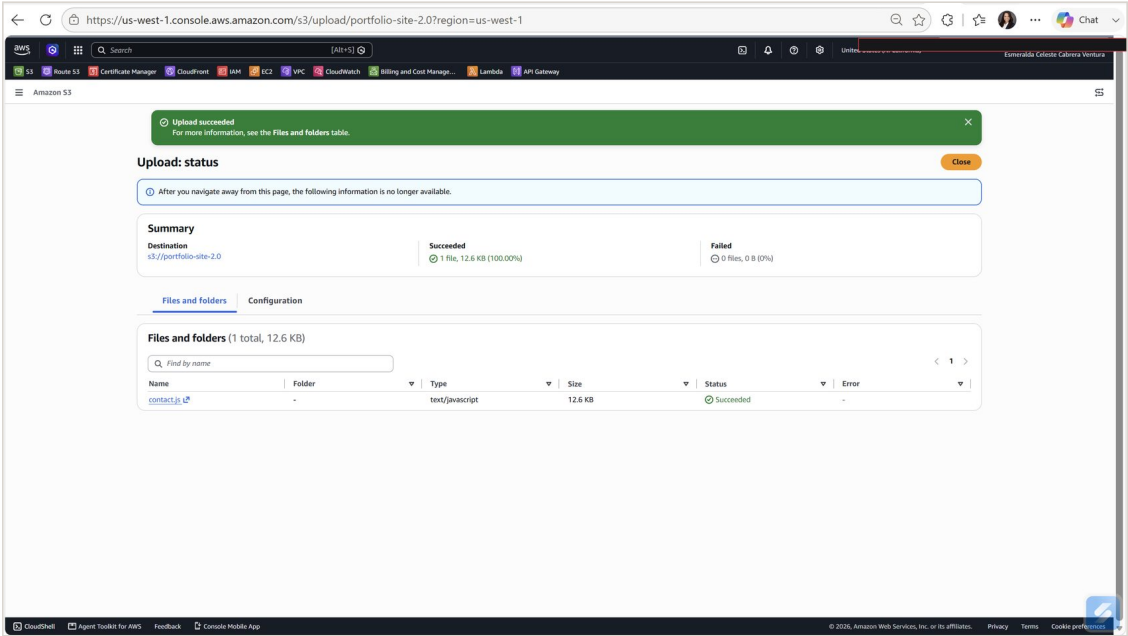


Figure 29. The updated client script uploaded to the site's S3 bucket.

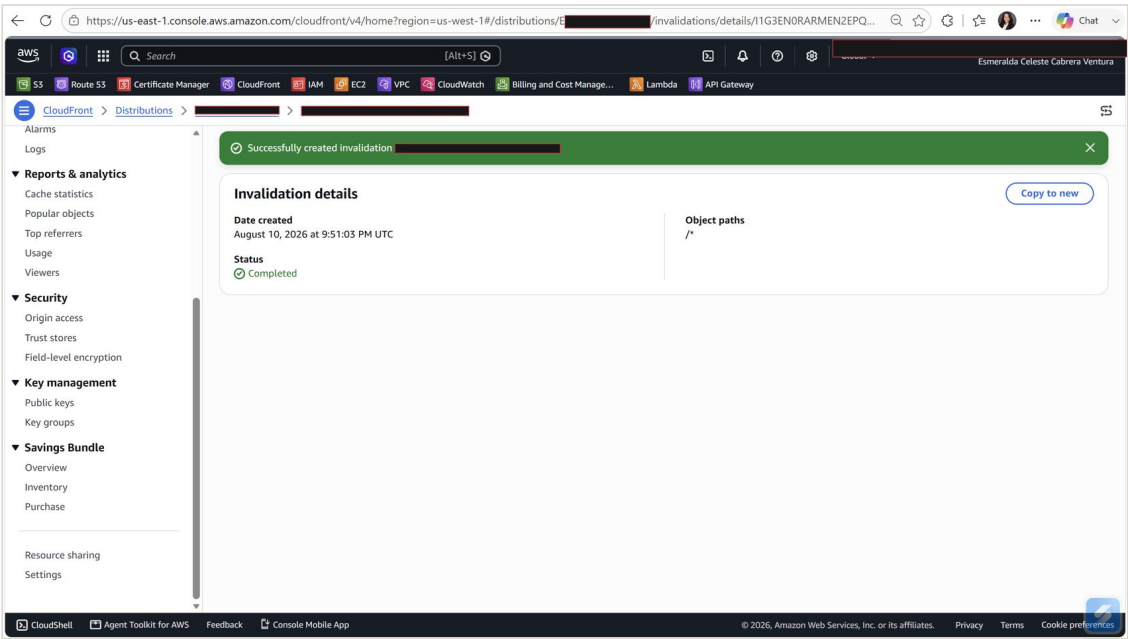


Figure 30. The invalidation completes, so the CDN now serves the new script.

5.7 Phase 7. Testing, failure and resolution

The first live submission fails. The cause is diagnosed from the browser console and corrected. The evidence that the corrected path actually delivers is presented in Section 10, where it belongs: it is the result of the project rather than a step in building it.

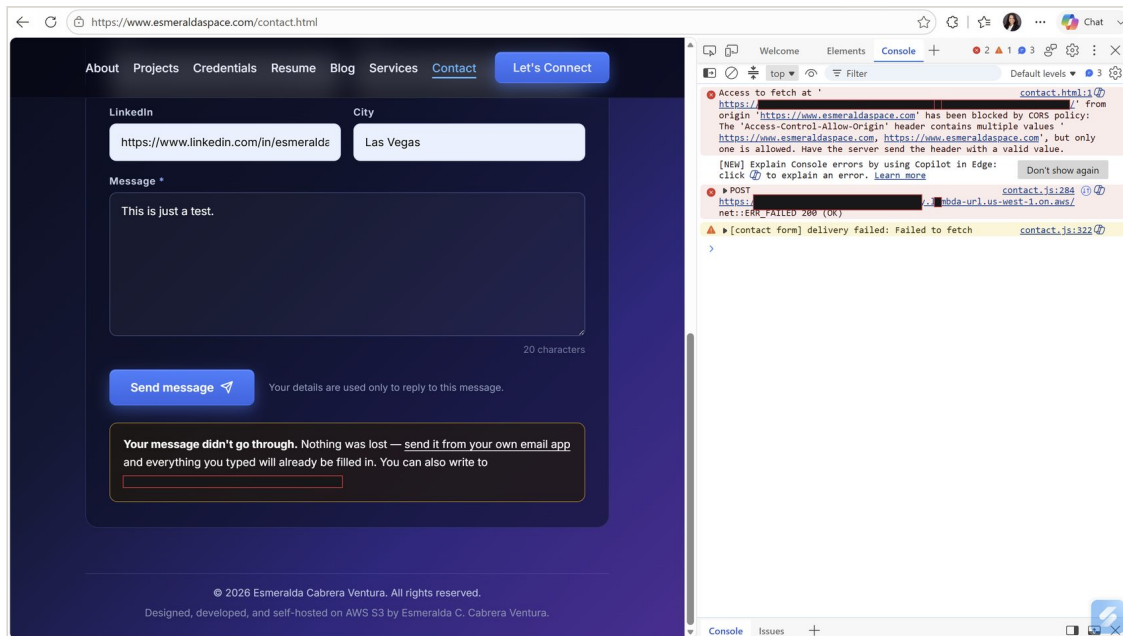


Figure 31. The first live submission fails. The browser console reports that the Access-Control-Allow-Origin header contains multiple values where only one is allowed, and the request shows a failed fetch alongside a 200 status. Below the form, the recovery path designed in Section 4.6 is working: the visitor is told the message did not go through, assured that nothing was lost, and offered a pre-filled draft and a direct address.

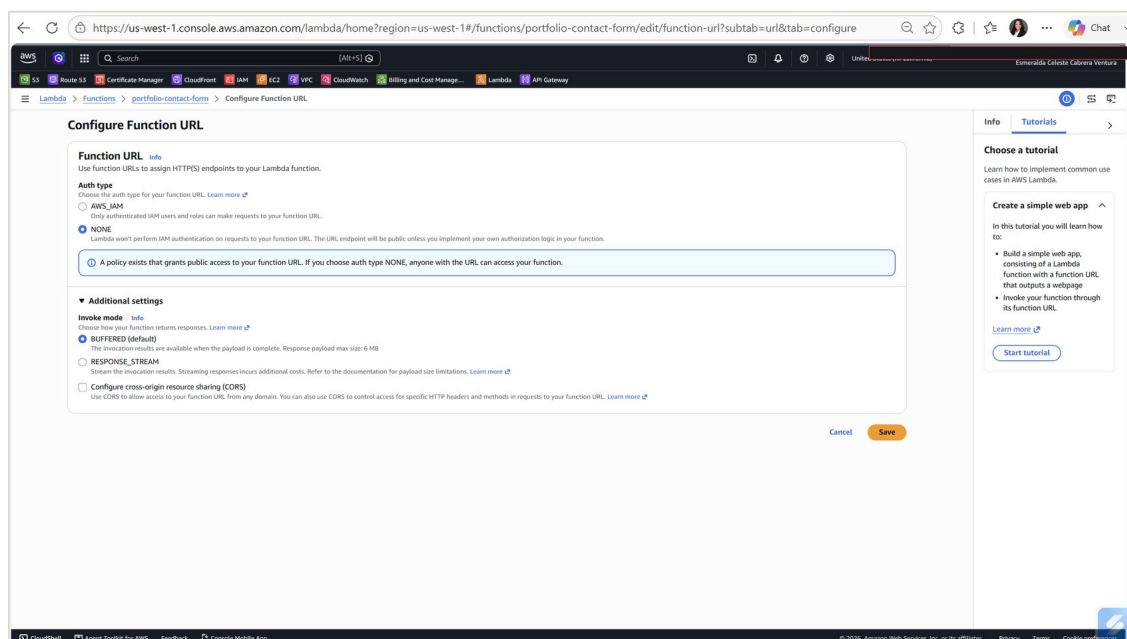


Figure 32. The fix. CORS is disabled on the Function URL, leaving the handler as the single owner of the header.

6. Incident analysis: the duplicated CORS header

The first live submission from the production site failed. The browser console reported that the request had been blocked by CORS policy because the Access-Control-Allow-Origin header contained multiple values where only one is permitted. The network entry showed a status of 200 alongside a failed fetch, which is exactly the confusing combination the design in Section 4.6 anticipated.

6.1 Root cause

CORS headers were being applied in two independent places. The Lambda Function URL has its own CORS configuration, and when it is enabled the Lambda service attaches the appropriate headers to the response as it passes through. The handler code was also constructing an Access-Control-Allow-Origin header from its `ALLOWED_ORIGIN` environment variable and returning it explicitly.

Neither layer was wrong in isolation. Both were doing precisely what they had been told to do. The response that reached the browser, however, carried the header twice. The CORS specification requires a single origin value, so the browser rejected the response and the fetch failed, even though the function had already run to completion and the request had returned 200.

6.2 Why the mistake was easy to make

This class of bug is characteristic of platform features that overlap with application code. The Function URL CORS panel presents itself as the way to configure CORS, and it is. The handler setting its own headers is also correct, and is what the code would need to do behind API Gateway or any other fronting service. The two are individually reasonable and jointly broken, and nothing in either interface warns about the other.

The diagnostic signal was precise and worth remembering. A browser error that names the header and says it contains multiple values is not describing a misconfigured origin. It is describing duplication, which means two things are setting the same header and one of them must stop.

6.3 Resolution

CORS was disabled on the Function URL, leaving the handler as the single authority for the header. Ownership in one place was chosen over ownership in the platform for two reasons. The handler already reads `ALLOWED_ORIGIN`, so the permitted origin stays in the same place as the rest of the configuration and can be changed without touching the URL settings. And the header logic travels with the code if the endpoint is ever moved behind API Gateway.

The reverse choice, deleting the header code and relying on the Function URL configuration, would have been equally valid. What matters is that exactly one layer owns the header.

6.4 What the incident cost

Nothing was lost. The failure occurred on a test submission, and the recovery path from Section 4.6 behaved as designed: the form reported the failure honestly and offered the fallback. Had the client been written to trust the 200 status, the same bug would have produced a form that appeared to work perfectly while discarding every message it received. That is the kind of defect that is discovered months later, by accident, when someone mentions they wrote and never heard back.

7. Security posture

The system holds no long-lived credentials. The Lambda function receives temporary credentials through its execution role, and that role can perform exactly one action beyond writing its own logs. There is no access key in the client, in the function source or in the environment variables. This is the single most important property of the design.

The endpoint itself is public. The Function URL uses an auth type of NONE, which means Lambda performs no authentication and the resource-based policy permits invocation by any principal. The CORS configuration restricts which origins a *browser* will allow to read the response; it does nothing whatsoever to stop a direct request from curl, a script or another server. Anyone who learns the URL can invoke the function.

This is an accepted risk rather than an oversight, and the blast radius was deliberately kept small. A successful invocation sends one email to a fixed address that the caller cannot influence, because the recipient comes from an environment variable rather than from the request. The realistic worst case is therefore inbox spam and a Lambda bill, not data disclosure or lateral movement.

It is still a gap, and the mitigations are listed in Section 9. The endpoint address has been redacted throughout this report for exactly this reason: publishing it would convert a theoretical exposure into an advertised one.

Practices worth carrying to other projects

No credentials in client code. One IAM action, not a managed policy. Configuration in environment variables rather than source. Explicit success reporting instead of trusting a status code. A designed failure path, not just a designed success path.

8. Operating cost

At portfolio volume the running cost is close to zero, and the figures below are the reason the architecture is worth the setup effort.

- **Lambda.** The perpetual free tier covers on the order of a million requests and 400,000 GB-seconds of compute per month. A contact form measured in tens of submissions sits far below any billable threshold.
- **SES.** Outbound email is billed per thousand messages, historically around ten cents per thousand for sending outside the free allowance. At this volume the cost rounds to nothing.
- **CloudFront and S3.** Already required to host the static site; the form adds one small JavaScript file and occasional cache invalidations.
- **CloudWatch Logs.** The only line item likely to grow unattended. Log groups are created with no expiry by default, so a retention policy is worth setting.

AWS pricing changes and varies by region, so these figures should be treated as an indication of magnitude rather than a quotation. The relevant conclusion is structural: this design has no idle cost, because nothing runs between submissions.

9. Known limitations and recommended next steps

The system works. The items below are the honest list of what is weak about it, in the order they should be addressed.

9.1 There is no abuse protection

A public, unauthenticated endpoint that sends email is an attractive target for automated abuse. The recommended mitigations, roughly in order of effort: set reserved concurrency on the function so that a flood cannot scale without limit; add a CAPTCHA or a proof-of-work challenge to the form; include a honeypot field that legitimate users never fill in; and apply simple per-IP rate limiting inside the handler. None is difficult, and the first is a single console setting that caps the worst-case bill.

9.2 The sending address undermines email authentication

This is the most consequential open issue. The FROM address belongs to a public mail provider whose domain the account does not control. SES cannot produce SPF or DKIM alignment for a domain it has no authority over, so the outgoing message fails DMARC alignment checks at the receiving end. The console warned about this during identity creation, and the deployed mail shows the symptom: recipients see the message annotated as sent *via* the SES domain, which is the mail client reporting exactly this mismatch. In practice such mail is more likely to be filtered as spam, and some strict recipients will reject it outright.

The fix is to verify the portfolio's own domain as an SES identity, publish the DKIM records SES provides, and send from an address at that domain such as `no-reply@` the site domain. The visitor's address should be placed in the Reply-To header rather than the From header, so that replying still reaches them while the envelope sender remains a domain the account controls. The function's own source comments already describe this as the correct approach, which makes it a known shortcut rather than an unknown flaw.

9.3 The IAM policy could be narrower

The policy grants `ses:SendEmail` against a resource wildcard. It can be scoped to the ARN of the specific verified identity, and further constrained with a condition on the permitted From address. That would reduce the compromised-function scenario from sending mail as any identity in the account to sending mail as one identity, from one address.

9.4 Operational gaps

- There is no alarm on function errors or on SES bounce and complaint rates. A CloudWatch alarm on the Errors metric is a few minutes of work and turns silent failure into a notification.
- CloudWatch log groups have no retention policy, so logs accumulate indefinitely.
- The runtime selected at creation time was a preview release. Preview runtimes are unsuitable for anything left running unattended and the function should be pinned to a generally available Node.js version.
- The handler's header comment describes the function as sitting behind API Gateway, which is no longer accurate. Stale comments are a small thing that cost real time later.
- The SES account remains in the sandbox. This is harmless while the only recipient is a verified address, but production access is a prerequisite for acknowledgement emails to visitors.

- Deployment is manual: edit the file, upload to S3, invalidate the CloudFront cache. This is acceptable at this scale and is the obvious first candidate for automation.

10. Closing assessment

The project achieved what it set out to achieve. A static site that previously could only offer a mailto: link now accepts messages and delivers them reliably, using infrastructure the site owner controls, at a cost that rounds to zero, with no credential stored anywhere in the system.

10.1 The result

The claim above is worth more with evidence behind it. Figure 33 is the outcome of the whole exercise: a message composed in a browser on a static website, carried through a public HTTPS endpoint, validated by a function holding no credentials, sent by a service that authenticated it through a role rather than a password, and delivered intact to the destination inbox. Every field submitted on the form arrives complete and correctly labelled, and the message body is preserved exactly as written.

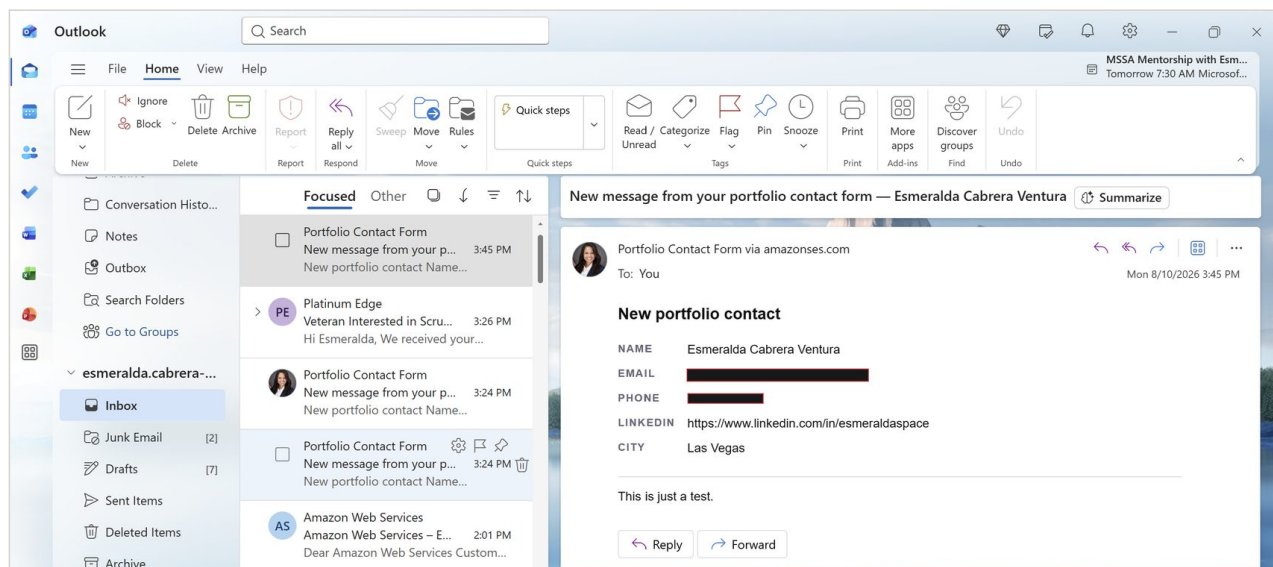


Figure 33. The result. A message submitted through the contact form on the live site, delivered to the destination inbox with every field intact and the body preserved exactly as written. This confirms the full path end to end: browser, Function URL, handler, SES, inbox. The sender annotation visible here, showing the mail as sent via the SES domain, is the one remaining flaw and is the symptom of the authentication alignment gap analysed in Section 9.2.

This is the same path that failed in Figure 31. The difference between the two figures is a single configuration change, and the fact that nothing was lost in between is a product of the failure handling described in Section 4.6 rather than of luck.

10.2 What holds up, and what does not

The engineering judgements that hold up best are the small disciplined ones: a single IAM action instead of a convenient managed policy, configuration lifted out of source into environment variables, and the decision not to treat an HTTP 200 as proof of delivery. That last one is what turned a production CORS bug from a

silent message-losing failure into a visible, recoverable error, and it is the decision most worth carrying into future work.

The weakest judgement is the sending identity. Verifying an address rather than a domain was the right way to get an end-to-end path working quickly, but it leaves the system's email authentication misaligned, and it is the first thing to change. Naming that clearly is more useful than a report which claims everything went perfectly.

Appendix A. Redaction methodology

Because this report is intended for publication, all thirty-three screenshots were processed before inclusion. Text was located by optical character recognition and each match was covered with an opaque box. Opaque boxes were used rather than blurring, since blurred text can sometimes be recovered computationally, whereas a filled rectangle discards the underlying pixels entirely.

Recognition was run at two scales over each image, plus a third higher-resolution pass over the console navigation strip, because character recognition accuracy varies with scale: small monospace text inside JSON policy documents was legible at one scale while code comments were legible only at another. Matching allowed for common recognition confusions between visually similar characters, and a geometric fallback covered the console account panel on the small number of screenshots where that text could not be read reliably at any scale.

The following categories were removed:

- The AWS account identifier, wherever it appeared, including inside every ARN.
- The IAM access key identifier and request signature contained in the SES verification emails.
- The Lambda Function URL hostname, which is an unauthenticated endpoint.
- Personal email addresses, a telephone number, and a mail thread identifier.
- The CloudFront distribution and invalidation identifiers.

Resource names that are not credentials were deliberately left visible, including the function name, the IAM role and policy names, the S3 bucket name and the site domain, since removing them would have made the walkthrough difficult to follow while protecting nothing. One leading digit of the account identifier remains legible in a single figure where the recognised text boundary fell fractionally short; a single digit of a twelve-digit identifier conveys no usable information.

A note on the source material

The verification emails in Figures 4 and 7 contained a live access key identifier and request signature in the confirmation link. Those links were single-use and have long since expired, so the practical exposure was negligible, but it is a useful reminder that automated notification emails routinely contain credential material and should be treated as sensitive when screenshots.